



HSE, VK

16/02/2026

Graph Neural Networks.

Alexander Tarakanov



Outline

From Matrix Factorization to Graph Neural Networks

How Graph Convolutional Networks Work

Oversmoothing in Graph Neural Networks

Oversquashing in Graph Neural Networks

Industry Application



Limitations of Matrix Factorization

Matrix Factorization models:

$$X \approx UV^T$$

Strengths

- Simple and scalable
- Strong baseline performance
- Low-dimensional latent representation

Limitations

- Ignores explicit graph structure
- Captures only pairwise user-item interaction
- No explicit modeling of higher-order relations
- Poor inductive generalization
- Struggles with sparse interaction graphs



Structural Blindness of Matrix Factorization

Collaborative signal often propagates along paths:

User A $\rightarrow$ Item 1 User B $\rightarrow$ Item 1 User B $\rightarrow$ Item 2 User C $\rightarrow$ Item 2

Matrix Factorization

- Captures direct interactions only
- Does not explicitly model multi-hop structure
- Global factorization may dilute local structure

Graph view

- Recommendation is inherently a graph problem
- Interaction matrix defines a bipartite graph



Why Graph Convolutional Neural Networks?

Core idea

$$H^{(k+1)} = \tilde{A}H^{(k)}W^{(k)}$$

Advantages over MF

- Explicit modeling of graph structure
- Natural multi-hop information propagation
- Incorporates node features and metadata
- Supports inductive learning (e.g., GraphSAGE)
- Flexible depth and propagation control

GNNs generalize collaborative filtering to structured message passing.



Why GNNs Work in Recommender Systems

User-item interactions define a bipartite graph.

Benefits

- Capture higher-order collaborative signals
- Model user and item neighborhoods explicitly
- Naturally extend to heterogeneous graphs
- Combine structural and content information
- Enable contrastive and self-supervised training

Example Models

- NGCF
- LightGCN
- GraphSAGE-based recommenders



Applications of Graph Neural Networks Beyond RecSys

Social Networks

- Community detection
- Link prediction
- Influence modeling

Biological Networks

- Drug discovery
- Protein interaction prediction
- Molecular property prediction

Knowledge Graphs

- Entity classification
- Relation prediction

Computer Vision

- Scene graphs
- 3D point cloud processing

GNNs are a general framework for learning on relational data.



Outline

From Matrix Factorization to Graph Neural Networks

How Graph Convolutional Networks Work

Oversmoothing in Graph Neural Networks

Oversquashing in Graph Neural Networks

Industry Application



Graph Convolution: Message Passing View

Given a graph with adjacency matrix A .

Add self-loops:

$$\hat{A} = A + I$$

Define normalized adjacency:

$$\tilde{A} = D^{-\frac{1}{2}} \hat{A} D^{-\frac{1}{2}}$$

GCN Layer

$$H^{(k+1)} = \sigma \left(\tilde{A} H^{(k)} W^{(k)} \right)$$

Interpretation

- $\tilde{A} H^{(k)}$: neighborhood aggregation
- $W^{(k)}$: feature transformation
- σ : nonlinearity



Spectral Interpretation of GCN

Graph Laplacian:

$$L = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

Spectral decomposition:

$$L = U \Lambda U^T$$

Spectral convolution

$$g_\theta * x = U g_\theta(\Lambda) U^T x$$

GCN approximation (Kipf & Welling):

$$g_\theta(\Lambda) \approx \theta(I + D^{-\frac{1}{2}} A D^{-\frac{1}{2}})$$

This leads to:

$$H^{(k+1)} = \tilde{A} H^{(k)} W^{(k)}$$

GCN = first-order approximation of spectral convolution.



Unified View of GCN

GCN can be interpreted as:

1. Message Passing

Local neighborhood aggregation.

2. Spectral Filtering

Low-pass filter on graph signals.

3. Diffusion Process

Repeated application:

$$H^{(K)} = \tilde{A}^K XW$$

Key Insight

GCN performs learnable graph diffusion followed by feature transformation.

GCN = Learnable Spectral Diffusion on Graphs



Outline

From Matrix Factorization to Graph Neural Networks

How Graph Convolutional Networks Work

Oversmoothing in Graph Neural Networks

Oversquashing in Graph Neural Networks

Industry Application



Oversmoothing in Graph Neural Networks

Phenomenon

As the number of GNN layers increases:

- Node embeddings become increasingly similar
- Representations collapse into a low-dimensional subspace
- Discriminative power decreases

Mathematical intuition

Repeated propagation:

$$H^{(k+1)} = \tilde{A}H^{(k)}W^{(k)}$$

acts as a **low-pass filter** on the graph.

In Recommender Systems

- Users become indistinguishable
- Items drift toward popularity bias
- Recommendation quality degrades



Spectral View of Oversmoothing

Let:

$$\tilde{A} = U\Lambda U^\top$$

Then:

$$\tilde{A}^k = U\Lambda^k U^\top$$

Observation

- Eigenvalues satisfy $|\lambda_i| < 1$
- High-frequency components decay faster
- Low-frequency components dominate

Consequence

$$H^{(k)} \rightarrow \text{smooth subspace}$$

Too many layers $\boxtimes$ excessive smoothing.



Architectural Solutions to Oversmoothing

1. Residual / Skip Connections

$$H^{(k+1)} = H^{(k)} + \text{GCN}(H^{(k)})$$

Preserves original information.

2. Initial Residual (APPNP-style)

$$H^{(k+1)} = (1 - \alpha)\tilde{A}H^{(k)} + \alpha H^{(0)}$$

Controls diffusion strength.

3. Jumping Knowledge (JK-Net)

Combine multiple layers:

$$H = \text{concat}(H^{(1)}, \dots, H^{(K)})$$

Preserves multi-scale information.



Regularization and Contrastive Approaches

DropEdge

Randomly remove edges during training.

- Reduces excessive diffusion
- Improves robustness

Contrastive Learning (SGL, SimGCL)

- Encourage diversity in embeddings
- Prevent representation collapse

Normalization (PairNorm)

Rescale embeddings to avoid collapse.



Oversmoothing in Recommender Systems

Most influential solutions in RecSys

- LightGCN (simplified propagation)
- Residual / PPR-style propagation
- Contrastive regularization (SGL, SimGCL)
- Layer combination strategies

Practical Insight

- 2–3 layers often sufficient
- More layers rarely improve performance
- Oversmoothing closely tied to popularity bias

Control diffusion — do not eliminate it.



Outline

From Matrix Factorization to Graph Neural Networks

How Graph Convolutional Networks Work

Oversmoothing in Graph Neural Networks

Oversquashing in Graph Neural Networks

Industry Application



Oversquashing in Graph Neural Networks

Definition

Oversquashing occurs when information from exponentially many distant nodes is compressed into fixed-size node embeddings during message passing.

Message passing

$$h_v^{(k+1)} = \text{AGG} \left(\{h_u^{(k)} : u \in \mathcal{N}(v)\} \right)$$

After K layers:

- Node v depends on all nodes within K -hop neighborhood
- Neighborhood size may grow exponentially
- Embedding dimension remains fixed

Too much information is forced through too small a bottleneck.



Structural Origin of Oversquashing

Example: Tree-like Graph

Branching factor = 3

After 5 hops:

$$3^5 = 243 \text{ nodes influence one root}$$

All signals must pass through:

- 5 aggregation layers
- fixed-size embedding vector

Key Insight

- Information grows exponentially
- Representation capacity is constant
- Information is compressed at every layer

Oversquashing is a **structural bottleneck** problem.



Oversquashing vs Oversmoothing

Aspect	Oversmoothing	Oversquashing
Core issue	Nodes become similar	Information bottleneck
Cause	Repeated averaging	Exponential neighborhood growth
Spectral view	Low-pass filtering	Structural congestion
Effect	Loss of discrimination	Loss of long-range signal

Intuition

- Oversmoothing: averaging too much
- Oversquashing: compressing too much



Oversquashing in Recommender Systems

Collaborative chains

User A $\rightarrow$ Item 1 User B $\rightarrow$ Item 1 User B $\rightarrow$ Item 2 User C $\rightarrow$ Item 2

Signal between A and C requires multi-hop propagation.

Problem

- Long-range collaborative effects are compressed
- Multi-hop signals vanish
- Deep GNN may behave like shallow model

Oversquashing limits the benefit of deeper GNN layers.



Modern Approaches to Mitigate Oversquashing

1. Architectural Improvements

- Residual / skip connections
- Larger embedding dimensions

2. Structural Encodings

- Positional embeddings
- Global structural features

3. Geometric / Sheaf-based Methods

- Replace scalar propagation with structured operators
- Preserve directional information

Oversquashing is a structural bottleneck — not just a depth problem.



Problem Statement.

Objective: recommend item(s) for a given user based on the current item and on the history.

Method: Graph Neural Networks (GNN) $\rightarrow$ user is represented via graph of viewed items.

Contribution:

- novel approach for graph construction
- novel GNN architecture
- novel approach for handling long-tail data



Outline

From Matrix Factorization to Graph Neural Networks

How Graph Convolutional Networks Work

Oversmoothing in Graph Neural Networks

Oversquashing in Graph Neural Networks

Industry Application



Graph Construction.

Connect item A with item B if user clicked on item B while viewing item A . Only strong action is utilized for denoising.

Deterministic algorithm for construction of edges based on user-item graph.
Implementation of Swing algorithm (based on the paper by guys from Alibaba).

Search Collaborative Filtering: items A and B appeared in the same search and user clicked on both items.

The main motivation for additional edges - graph sparsity reduction.



Neural Network Architecture.

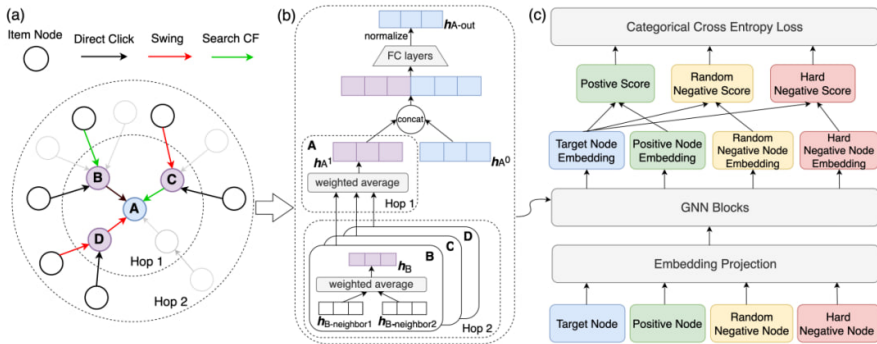


Figure 1: Overview design of our GNN: (a) Logic of edge formation and neighbor sampling (b) Aggregation logic of the GNN block model (c) The overview architecture and training flow of LightSAGE

Lower number of non-linear activation functions (PinSAGE), lower requirements for memory (LightGCN and GAT).



Model Training.

Sample item A - target node.

Sample k items from A via random walk algorithm - this subgraph is a batch.

Sample one random neighbor for each item.

Generate negative samples: degree-based algorithm and batch-based algorithm.

Compute embeddings $\rightarrow$ cosine measure w.r.t target node $\rightarrow$ learn classifier based on cross-entropy loss.



Long-Tail Items.

Long-Tail items - items that have low probability to appear in the search.

Average embedding vectors of similar (brand, price and etc..) long-tail items.

Utilize averaged embedding for training.

Inference: if long-tail item is a neighbor of the target item - do not perform averaging - use as it is.



Performance.

Offline test and online A/B test.

Table 1: Offline performance of LightSAGE as compared to other architectures.

Model	AUC	Unique recall	Tail unique recall
Node2vec	0.872	+0.00%	+0.00%
PinSAGE	0.882 (+1.02%)	+0.81%	+0.29%
GAT	0.878 (+0.62%)	+0.79%	-1.44%
LightSAGE	0.890 (+1.92%)	+1.42%	+1.29%

Table 2: Impact of improvement made in terms of model, graph construction, and long-tail handling.

Model	Graph	Tail handling	Unique recall	Tail unique recall
Node2vec	Click sequences	No	+0.00%	+0.00%
LightSAGE	Click sequences	No	+ 0.99%	+0.20%
LightSAGE	Direct clicks	No	+70.20%	+13.45%
LightSAGE	Direct clicks + CF	No	+81.90%	+112.37%
LightSAGE	Direct clicks + CF	Content	+74.39%	+235.93%
LightSAGE	Direct clicks + CF	Content + graph	+88.40%	+303.29%

Orders and revenue: +1% and +5%.

Number of unique ads that have users' impressions and clicks: +10% and +3%.

Deployed!